CS110 / INDEPENDENCE ROAD

reportB / Lexxie

Issued: Nov 12, 2026, 1:52 PM Pacific

Record: FORM-957B214A857BA6296BF323AB

Personal public verification code: VIEW-9265F91D9B43EF8355AB19A9

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: 196722470cb2aabde13c7b6cd34d1ef101a1c8f5eea50e21946c8c397b7b50ba

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

What did you personally execute? Connect completed tasks to files/commits.

What I personally did: I'm working solo, so everything here is my own work. I planned the StimuLye vertical slice and wrote the pitch, mission, tool stack and a five-stage build pipeline (Setup, NaN avatar, ward prefabs, Stimuli mechanic, playtest). I made two documentation boards, a team plan board and a checkpoint board, and wrote my recap and huddle review notes.

Evidence: Is inside the github folder through the previous actions.

What I observed: When I opened the boards, the five stages connected in order and each had a clear test. The checkpoint board shows planning at 100% and the Unity build at 0%, which matches where the project actually is.

Unresolved problem / question: I have not built the scene in Unity yet, so I can't show test results for movement, the avatar, or the Stimuli mechanic. My next step is stage 1, creating the Unity 6.6 project with the Third Person Starter Assets and testing WASD, jump and camera. I also still need to confirm how detailed "beautiful" needs to be for this assignment.

Record setup/import checks, project opens/runs, player configuration/result, and one failure fix retest. Separate observed results from predictions.

Setup / import checks: I created the project from Unity 6.6's High Definition 3D (HDRP) template. The render pipeline and sample scene were already configured, so I did not need to import packages manually. Observed: the Console showed 0 errors after opening.

Player configuration and result: Observed: the sample scene loads and I can move through it / the third-person player moves with WASD, jumps and the camera follows.

Failure fix retest: No failure occurred, because the template was pre-configured, so I have no fix to report yet.

Predictions (not observed yet): HDRP is heavier than URP, so frame rate may drop on my computer once I add my own scene. Mixamo materials may show pink in HDRP, and I would fix that with the Render Pipeline Converter or by assigning HDRP/Lit materials, then retest.

Personal GitHub URL/commit, team link if used, project/source paths, PDFs, boards/images, credits and reproduction instructions.

Personal GitHub: https://github.com/Lexicon-Lexxie/CS110-Lexxie (folder Week12/12B).

CS110 / INDEPENDENCE ROAD

Commit:https://github.com/Lexicon-Lexxie/CS110-Lexxie/commit/609ae457fe348d2259576d1b180096a9fe684ea7
Solo project, so there is no team link.

Project / source paths: The Unity 6.6 (6000.6.7f1) HDRP project is at Week12/12B/project-source/UnityProject/. The scene is Assets/Scenes/SampleScene.unity. My own work goes in Assets/StimuLye/ (Scripts, Prefabs, Materials, Models, Animations, Audio, Scenes); these folders are set up but empty so far, because I have not started the custom build.

PDFs: Week12/12B/Requirements/issued-pdfs/ (PREP-F39AC39A936F4F003C2A6D22 and FORM-2935112442B32DC30DC60A17, FORM-9A9193840307A5B39A239303, FORM-A91B6BD4CBEDB5F72519F72E). Working-Draft.pdf is in Requirements/ and is not an issued record.

Boards / images: Resources/boards/ (prepBoard, pipelineBoard, huddleBoard, reportABoard) and test evidence in Evidence/testfixconsole.png, a Play-mode screenshot with the Console.

Credits: The scene, third-person controller, meshes, materials and textures come from Unity's High Definition 3D (HDRP) template (Assets/SampleSceneAssets/), so they are Unity's, not mine. I have added no other third-party assets yet. Any future outside asset (e.g. Mixamo model, audio) will be credited in Requirements/Credits.txt.

How to reproduce: (1) Clone the repo. (2) In Unity Hub, Add project from disk and choose Week12/12B/project-source/UnityProject/. (3) Open with Unity 6000.6.7f1; the first open is slow while Library/ regenerates. (4) Open Assets/Scenes/SampleScene.unity and press Play. (5) Move with WASD, sprint with Shift, jump with Space. Full folder map and instructions are in Week12/12B/README.md. The report opens by double-clicking index.html.

Observed: In my test the project opened and ran in Play mode with the third-person player visible and the Console showing 0 errors and 1 warning (HDRP "ScriptableRuntimeReflectionSystemSettings.system is assigned more than once", a template warning). Not yet tested: NaN import, ward prefabs, Stimulus scripts, frame rate.

Personal GitHub: https://github.com/Lexicon-Lexxie/CS110-Lexxie (folder Week12/12B). Commit: [paste the commit URL or short hash]. Solo project, so there is no team link.

Project / source paths: The Unity 6.6 (6000.6.7f1) HDRP project is at Week12/12B/project-source/UnityProject/. The scene is Assets/Scenes/SampleScene.unity. My own work goes in Assets/StimuLye/ (Scripts, Prefabs, Materials, Models, Animations, Audio, Scenes). These folders are set up but empty so far, because the custom build has not started.

PDFs: Week12/12B/Requirements/issued-pdfs/ (PREP-F39AC39A936F4F003C2A6D22, FORM-2935112442B32DC30DC60A17, FORM-9A9193840307A5B39A239303, FORM-A91B6BD4CBEDB5F72519F72E). Working-Draft.pdf is in Requirements/ and is not an issued record.

Boards / images: Resources/boards/ (prepBoard, pipelineBoard, huddleBoard, reportABoard). Test evidence is Evidence/testfixconsole.png, a Play-mode screenshot with the Console.

Credits: The scene, third-person controller, meshes, materials and textures come from Unity's High Definition 3D (HDRP) template (Assets/SampleSceneAssets/), so they are Unity's, not mine. I have added no other third-party assets yet. Any future outside asset (such as a Mixamo model or audio) will be credited in Requirements/Credits.txt.

Reproduction instructions: (1) Clone the repo. (2) In Unity Hub, choose Add project from disk and select Week12/12B/project-source/UnityProject/. (3) Open it with Unity 6000.6.7f1; the first open is slow while Library/ regenerates. (4) Open Assets/Scenes/SampleScene.unity and press Play. (5) Move with WASD, sprint with Shift, jump with Space. The full folder map is in Week12/12B/README.md, and the report opens by double-clicking index.html.

Observed (Unity): The project opened and ran in Play mode with the third-person player visible. The Console showed 0 errors and 1 warning (HDRP "ScriptableRuntimeReflectionSystemSettings.system is assigned more than once", which comes from the template). Not yet tested: NaN import, ward prefabs, Stimulus scripts, frame rate.

Failure -> fix -> retest (Git push): When I pushed my reorganized folder, GitHub rejected it with error GH001 because a 209 MB file, Week12/12B.zip, was in my commit and GitHub's limit is 100 MB. That zip was only a transfer copy and should never have been committed. Deleting the file in a later commit did not help, because the zip was still in the unpushed history. Fix: I ran git reset --soft origin/main to undo my two unpushed commits while keeping my files, removed the zip from Git and from disk, added *.zip to .gitignore, and checked with git status that no file over 100 MB was staged. Retest: I committed again and the push succeeded. The largest remaining files are two template lightmaps (about 27 MB and 23 MB), which are under the limit. Lesson: check git status for large files before committing.

Other issue noted: my earlier planning notes say URP, but the project uses HDRP. I documented this in README.md section 9.

Compare the result with the mission. What did you verify, what remains, and what should the team do next?

Mission: Make a beautiful playable scene in Unity with a 3D player viewed in third person, built as a StimuLye vertical slice (dark opening, looping ward corridor, one Stimuli memory room).

What I verified (observed): The Unity 6.6 (6000.6.7f1) High Definition 3D template opens and runs in Play mode. The third-person player is already in the scene and controllable (WASD, Shift, Space), and the Console showed 0 errors and 1 warning (HDRP "ScriptableRuntimeReflectionSystemSettings.system is assigned more than once", from the template). Evidence: Evidence/testfixconsole.png. I also organized the repo (Context, Requirements, Resources, Evidence, project-source), wrote a README with all paths and reproduction steps, and pushed it to GitHub after fixing a push rejection caused by a 209 MB zip that I had committed by mistake.

Useful discovery: The template already provides the third-person player, camera and a high-quality lit scene, so the "beautiful scene + third-person player" part of my goal works before I build anything custom. This changes my plan: instead of spending time on a controller, I should spend it on the StimuLye content. I also discovered that my planning notes assumed URP, but the template is HDRP, which is heavier and means my C# package table (UnityEngine.Rendering.Universal) is wrong for this project. Finally, I learned to check git status for large files before committing, because GitHub rejects files over 100 MB.

CS110 / INDEPENDENCE ROAD

What remains (not yet done or tested): importing the NaN model (Mixamo, Humanoid rig) into PlayerArmature; building modular ward and corridor prefabs in Assets/StimuLye/Prefabs; writing StimulusItem and RoomStabilizer in Assets/StimuLye/Scripts; adding the Static audio cue; and measuring frame rate and console errors in a final playtest. The Assets/StimuLye folders exist but are empty.

Next steps: (1) decide whether to keep HDRP or switch to URP, and update the documentation to match, (2) import NaN and test the swap, which is where I expect my first failure, fix and retest, (3) whitebox one corridor and one memory room with prefabs, (4) write the Stimulus scripts and test that pickup changes the room, (5) playtest for 0 errors and stable FPS and record the result in Project_LOG.txt.

Unresolved question: Is HDRP acceptable for this assignment, or should I use URP as I planned? I will keep HDRP unless told otherwise, because the template already works.

Name

Lexxie

Action

B

Issued team form

FORM-2935112442B32DC30DC60A17

Whiteboard

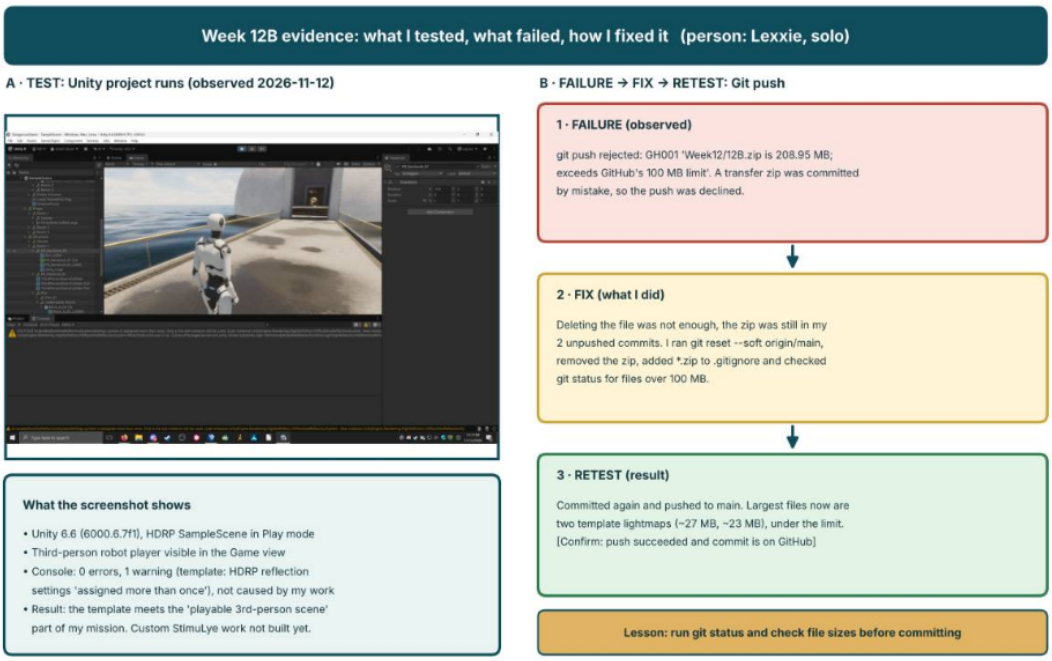
Person involved: Lexxie (solo project, so all work and decisions are mine).

What this image shows: Left side (A) is my real Unity test from 2026-11-12. Unity 6.6 (6000.6.7f1) opened the HDRP SampleScene and ran in Play mode. The third-person robot player is visible in the Game view, and the Console shows 0 errors and 1 warning. The warning says an HDRP reflection setting is "assigned more than once". It comes from the template, not from my changes. Result: the template already gives me a playable third-person scene, which is the first part of my mission. My own StimuLye content (NaN model, ward prefabs, Stimulus scripts) is not built yet.

Right side (B) is a failure, fix and retest from my Git work. Failure: my push was rejected by GitHub (error GH001) because a 209 MB file, Week12/12B.zip, was in my commit and the limit is 100 MB. I had only used the zip to transfer files and committed it by mistake. Fix: deleting the file was not enough because it was still in my two unpushed commits. I ran git reset --soft origin/main to undo the commits while keeping my files, removed the zip, added *.zip to .gitignore, and checked git status for any file over 100 MB. Retest: I committed and pushed again, and the push went through. The largest files left are two template lightmaps of about 27 MB and 23 MB, which are under the limit.

My interpretation: the template saves me from building movement and camera, so my time should go into the StimuLye content. The Git problem showed me to check file sizes before committing, because large files block the whole push.

CS110 / INDEPENDENCE ROAD



Member time receipts

0

Name

Lexxie

verificationCode

VIEW-9265F91D9B43EF8355AB19A9

A

Time code

TIME-096E904E83B4BD764B9404FD

Name

Lexxie

Action

A

Server arrival

2026-11-10T21:02:47.141Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

15

Late deduction points

15

Review status

Calculated awaiting instructor review

CS110 / INDEPENDENCE ROAD

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

B

Time code

TIME-0CB7A885F2F8DC76AFAA8623

Name

Lexxie

Action

B

Server arrival

2026-11-12T20:46:44.620Z

Class starts

2026-11-12T20:45:00Z

Calculated late points

0

Late deduction points

0

Review status

Calculated awaiting instructor review

recovery

Review status

Not requested

requestedAt

restoredPoints

0

approvedAt

workNotes

Task list

0

CS110 / INDEPENDENCE ROAD

id

75169224-740e-41a8-8b8a-24a6137bee77

task

Documentation

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

1

id

de1342be-2e15-45b8-a532-e54167c65a25

task

Setup

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

2

id

ea92e30c-008a-4533-9ce6-e2786e733ed4

task

Player import

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

3

CS110 / INDEPENDENCE ROAD

id

3ba2fd21-ccbe-4d2d-8b62-3282e36cbb0b

task

Testing

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

4

id

66088a36-ec28-4a94-a6d7-d33cb121cf18

task

Handoff

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

reviewedPlan

mission

I want to make a beautiful playable scene in unity with a 3D player viewed in 3rd person.

framework

I will use the AAA framework in order to keep an organized structure of how I am able to complete my project efficiently.

pipeline

Unnamed stage

Unnamed stage

I used mermaid owner Lexxie

Mermaid worked better for me.

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

CS110 / INDEPENDENCE ROAD

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.
- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)
- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)

tech

What I Did / Decided / Observed:

As a solo developer on StimuLye, my priority is eliminating pipeline friction so I don't get overwhelmed wearing every hat alone. I applied the TnT ("Tool and the Temple") principle: every tool must serve my MVP vertical slice and reduce manual labor. I chose a lean, verified stack built on Unity 6.6 and established personal testing gates to check my own work at each milestone.

My Agreed Solo Tool Stack & Verification Plan:

Engine: Unity 6.6 (6000.6) with URP

Why: Faster Play Mode entry, lighter memory footprint, and native Universal Render Pipeline lighting/fog to achieve

CS110 / INDEPENDENCE ROAD

StimuLye s dark atmosphere without third-party render plugins.

Who Checks It: Me (wearing the Technical Hat) verified via build tests and frame rate checks.

Language & IDE: C# / Visual Studio Community

Why: Standard Unity scripting for clean, fast implementation of the "Stimuli" interaction and room material swap trigger scripts.

Who Checks It: Me (wearing the Programmer Hat) verified with zero console errors and clean syntax.

Base Template: Starter Assets Third Person Controller + Cinemachine

Why: Provides an instant movement, sprint, jump, and camera follow baseline. As a solo dev, this saves me dozens of hours of physics and camera programming so I can focus directly on the story mechanics.

Who Checks It: Me (wearing the Technical Hat) verified through Play Mode controller feel.

Art & Rigging: Mixamo + Blender

Why: Mixamo automatically generates the Humanoid rig and Avatar for my custom "NaN" character; Blender is used for quick adjustments and whiteboxing modular ward prefabs.

Who Checks It: Me (wearing the Visual Hat) verified by checking that animations retarget without bone stretching.

Audio: Audacity / FreeSound (WAV stems)

Why: Simple, lightweight sound editing for hospital room tone, heart monitor beeps, and the "Static" cue.

Who Checks It: Me (wearing the Audio Hat) verified with in-engine volume balance checks.

Progress Tracking: Git + Project_LOG.txt

Why: Local Git commits protect my project from corruption; the auto-timestamped .LOG file holds me accountable to daily tasks.

Who Checks It: Me (wearing the QA / Architect Hat) audited against my GDD checklist.

Question / Self-Check:
How can I automate my daily .LOG entries and Git commits so project tracking remains effortless while working solo?

CS110 / INDEPENDENCE ROAD

Time code

Classes & components

Class | Responsibility | GameObject

ThirdPersonController | Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters. | PlayerArmature

StarterAssetsInputs | Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags. | PlayerArmature

CharacterController | Unity engine physics component that manages capsule collision, ground detection, and slopes. | PlayerArmature

StimulusItem | Detects player proximity, triggers interaction events, and signals memory recovery when collected. | Stimulus_MemoryObject

RoomStabilizer | Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures. | Environment_Manager

StaticController | Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks. | Static_Antagonist

PauseMenuController | Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene. | UI_Canvas_Manager

Variables / properties

Name : type | Value / purpose | Used by

MoveSpeed | Defines base walk velocity vs. sprint speed in the ward. | ThirdPersonController

_animIDSpeed | Cached hash IDs for fast Animator parameter updates. | ThirdPersonController

isStabilized | Prevents the room transformation from firing more than once. | RoomStabilizer

darkMaterial | References to the low-poly void shader vs. lit hospital texture. | RoomStabilizer

staticProximityThreshold | Distance from player before static distortion audio begins. | StaticController

stimulusSound | Audio stems played on memory pickup and enemy approach. | AudioManager

f(x) Functions

Function | Inputs result | When called

Awake() | Engine Initialization | Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

Start() | First Frame | Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

Update() | Every Frame | Polls player movement vectors, updates camera angles, and checks for the Escape key input.

OnTriggerEnter(Collider) | Physics Collision Event | Detects when NaN steps into the StimulusItem radius or enters a "Static" trigger volume.

OnFootstep(AnimationEvent) | Animation Event | Plays footstep sound cues synchronized with the walk/run animation clip frames.

Packages / using

Package / namespace | Why included | Tested

using UnityEngine; | The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types. | [x]

using UnityEngine.InputSystem; | Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag. | [x]

using Cinemachine; | Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement. | [x]

using UnityEngine.Events; | Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other. | [x]

using UnityEngine.Rendering.Universal; | Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect. | [x]

communication

Team check-ins

Description / contact / owner | When Pacific | Agreed

Lexxie | | [x]

reuse

What I Did / Decided / Observed:

Thinking beyond this initial prototype assignment, I audited our production assets to determine what foundational systems can be reused across Act 2 (The Ward), Act 3 (The Deep), and the Finale without rebuilding from scratch. I decided to maintain a strict licensing and attribution log in /Requirements/ so all third-party credit requirements are tracked before scaling the game.

Reusable Components, Sources & Credit Requirements:

Core Character Controller & Camera Rig:

What is Reusable: The PlayerArmature with Third-Person movement physics, jump/sprint logic, and Cinemachine orbital follow camera.

Source: Unity Technologies (Starter Assets Third Person Character Controller).

Credit / License: Unity Companion License / Asset Store Terms. Free for commercial and non-commercial compiled builds; requires standard Unity engine inclusion in game credits.

Protagonist Model & Rigging:

What is Reusable: The Humanoid bone hierarchy, animation controller mapping, and custom "NaN" mesh.

Source: Adobe Mixamo (Erika_Archer base mesh & standard Humanoid locomotion clips).

Credit / License: Royalty-free under Adobe Mixamo Terms of Service for use in embedded video game builds

CS110 / INDEPENDENCE ROAD

(attribution to Adobe Mixamo in credits; raw .fbx files cannot be redistributed open-source).

Modular Room & Corridor Prefab Kit:

What is Reusable: Floor, wall, and doorway prefabs that can be re-skinned into decaying Backrooms/liminal spaces for Act 3 using shader/material swaps.

Source: Custom-modeled in Blender / Unity primitives.

Credit / License: Original creator IP 100% owned, no third-party attribution required.

"Stimuli" Trigger & Room Stabilization Scripts:

What is Reusable: The event-driven C# interaction architecture (StimulusItem.cs, RoomStabilizer.cs) can power every memory puzzle across all remaining chapters.

Source: Custom original C# codebase.

Credit / License: Original creator IP fully reusable across future episodes or spinoff projects.

Audio & Distortion Cues:

What is Reusable: Heartbeat pulses, hospital ambient drone, and the "Static" entity audio stems.

Source: Pixabay / FreeSound.org.

Credit / License: Pixabay Content License / Creative Commons (CC-BY 4.0 / CC0). Requires listing contributing sound artist handles in the game's final end-credits roll.

Question / Next Step:

Should I set up an automated Credits.txt file inside /Requirements/ now so that attribution for new sound stems and models is recorded the moment they are downloaded?

boardNotes

I will continue to work on the small project until I am ready to move up to the team project.

Whiteboard

Diagram: StimuLye vertical slice plan (solo, Unity 6.6, AAA framework)

The mission is a playable vertical slice: a dark opening, one looping ward corridor, and one Stimuli memory room. I'm working solo, so each box shows the "hat" I wear.

CS110 / INDEPENDENCE ROAD

Stage 1 (Application, Technical hat): set up the project with Unity 6.6 + URP and the Third Person Starter Assets so I have working movement immediately. I also create the folders and Project_LOG.txt. Test: WASD, jump, and camera work.

Stage 2 (Action, 3D Artist hat): import the Mixamo NaN model, set it to Humanoid, unpack PlayerArmature, and swap the geometry. Test: it runs and jumps with no pink shaders.

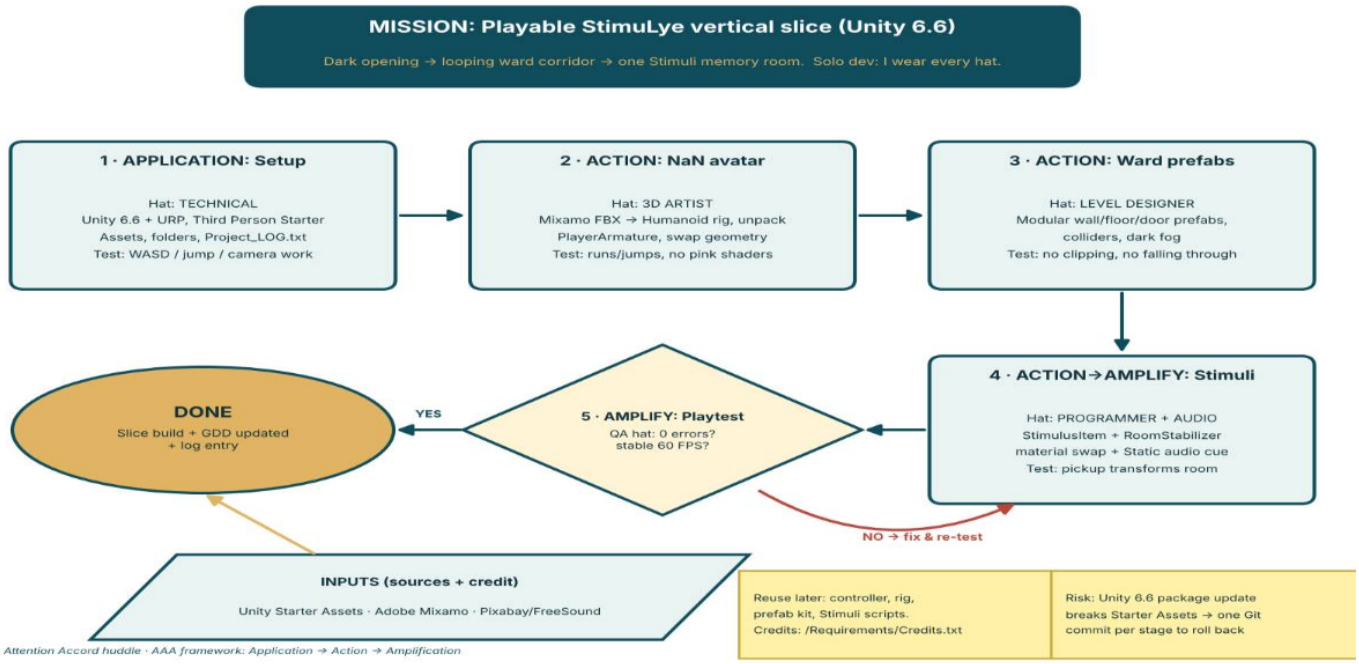
Stage 3 (Action, Level Designer hat): build modular wall, floor, and door prefabs with colliders and dark fog. Test: no clipping or falling through floors.

Stage 4 (Action to Amplification, Programmer + Audio hat): StimulusItem triggers RoomStabilizer, which swaps dark materials for hospital ones and plays the Static audio cue. Test: pickup visibly transforms the room.

Stage 5 (Amplification, QA hat): the decision diamond. If the playtest shows errors or low FPS, I loop back to stage 4 and fix. If it is clean (0 errors, stable 60 FPS), the slice is Done and I update the GDD and log.

Inputs are Unity Starter Assets, Adobe Mixamo, and Pixabay/FreeSound. Every credit goes in /Requirements/Credits.txt so these parts can be reused in later acts.

Risk: a Unity 6.6 package update could break Starter Assets, so I make one Git commit per stage to roll back.



Whiteboard

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's

verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

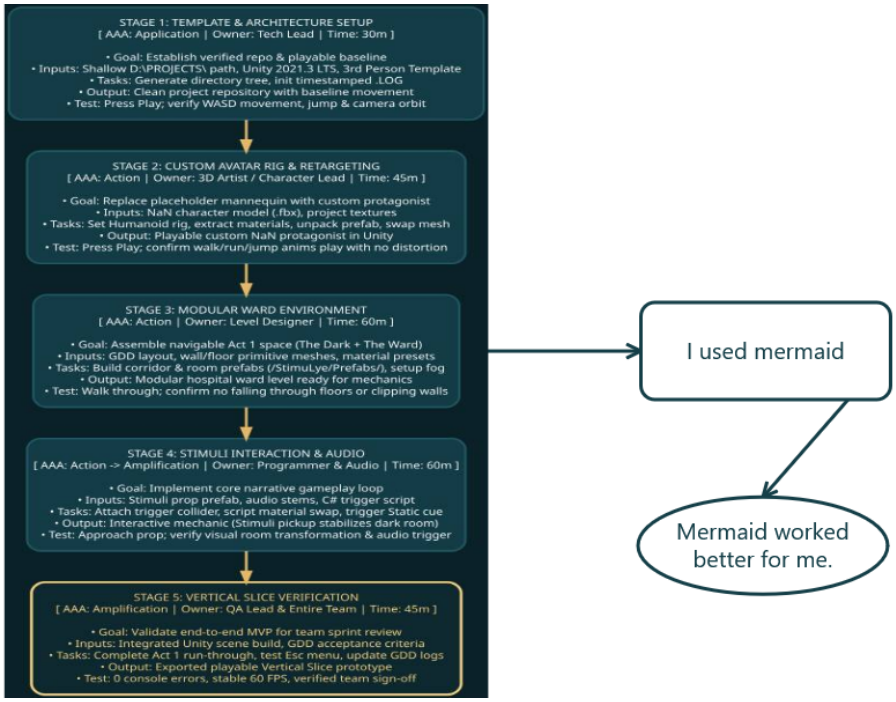
- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.
- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)
- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)

CS110 / INDEPENDENCE ROAD



codeTables

0

id

061b1987-dcfd-45cf-80bc-0948d0a0ace7

title

Classes & components

columns

0

label

Class

type

text

1

label

Responsibility

type

text

2

label

GameObject

type

text

rows

CS110 / INDEPENDENCE ROAD

0

0

ThirdPersonController

1

Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters.

2

PlayerArmature

1

0

StarterAssetsInputs

1

Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags.

2

PlayerArmature

2

0

CharacterController

1

Unity engine physics component that manages capsule collision, ground detection, and slopes.

2

PlayerArmature

3

0

StimulusItem

1

Detects player proximity, triggers interaction events, and signals memory recovery when collected.

2

Stimulus_MemoryObject

4

0

RoomStabilizer

1

Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures.

2

Environment_Manager

5

0

StaticController

1

CS110 / INDEPENDENCE ROAD

Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks.

2

Static_Antagonist

6

0

PauseMenuController

1

Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene.

2

UI_Canvas_Manager

1

id

6a62977a-857f-4e05-b30d-ec5a07798942

title

Variables / properties

columns

0

label

Name : type

type

text

1

label

Value / purpose

type

text

2

label

Used by

type

text

rows

0

0

MoveSpeed

1

Defines base walk velocity vs. sprint speed in the ward.

2

ThirdPersonController

CS110 / INDEPENDENCE ROAD

1

0

_animIDSpeed

1

Cached hash IDs for fast Animator parameter updates.

2

ThirdPersonController

2

0

isStabilized

1

Prevents the room transformation from firing more than once.

2

RoomStabilizer

3

0

darkMaterial

1

References to the low-poly void shader vs. lit hospital texture.

2

RoomStabilizer

4

0

staticProximityThreshold

1

Distance from player before static distortion audio begins.

2

StaticController

5

0

stimulusSound

1

Audio stems played on memory pickup and enemy approach.

2

AudioManager

2

id

595bbdd0-1eb9-4603-a8b8-c8d31f390b23

title

CS110 / INDEPENDENCE ROAD

f(x) Functions

columns

0

label

Function

type

text

1

label

Inputs result

type

text

2

label

When called

type

text

rows

0

0

Awake()

1

Engine Initialization

2

Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

1

0

Start()

1

First Frame

2

Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

2

0

Update()

1

Every Frame

2

Polls player movement vectors, updates camera angles, and checks for the Escape key input.

CS110 / INDEPENDENCE ROAD

3

0

OnTriggerEnter(Collider)

1

Physics Collision Event

2

Detects when NaN steps into the StimulusItem radius or enters a "Static" trigger volume.

4

0

OnFootstep(AnimationEvent)

1

Animation Event

2

Plays footstep sound cues synchronized with the walk/run animation clip frames.

3

id

b0f46e68-aa40-4beb-a479-052e7cd5204d

title

Packages / using

columns

0

label

Package / namespace

type

text

1

label

Why included

type

text

2

label

Tested

type

checkbox

rows

0

0

using UnityEngine;

CS110 / INDEPENDENCE ROAD

1

The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types.

2

true

1

0

using UnityEngine.InputSystem;

1

Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag.

2

true

2

0

using Cinemachine;

1

Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement.

2

true

3

0

using UnityEngine.Events;

1

Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other.

2

true

4

0

using UnityEngine.Rendering.Universal;

1

Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect.

2

true

communicationTables

0

id

CS110 / INDEPENDENCE ROAD

5a24f4fd-3441-4b83-8102-50851844f53a

title

Team check-ins

columns

0

label

Description / contact / owner

type

text

1

label

When Pacific

type

datetime-local

2

label

Agreed

type

checkbox

rows

0

0

Lexxie

1

2

true

Preparation record

PREP-F39AC39A936F4F003C2A6D22

Contribution record

FORM-A91B6BD4CBEDB5F72519F72E

dayBReview

goal

true

owners

true

blockers

true

notes

CS110 / INDEPENDENCE ROAD

Solo review, so I checked each statement myself.

Goal and finish line: I'm aiming for a beautiful, playable Unity scene with a 3D third-person player. My finish line is one working vertical slice (dark opening, looping ward corridor, one Stimuli memory room) that plays with no console errors.

Next task, deadline and dependencies: My next task is stage 1, setting up the Unity 6.6 project with the Third Person Starter Assets and Project_LOG.txt. Each later stage depends on the one before it: avatar, then ward prefabs, then the Stimuli scripts, then the playtest. I can't start the Stimuli scripts until the avatar and prefabs both work. I'll finish one stage per work session and keep one Git commit per stage.

Blockers and how I'll check results: The main blocker is a Unity 6.6 package update breaking the Starter Assets, which Git rollbacks cover. I'll check each stage by running it in Play mode (WASD, jump, camera, no pink shaders, no clipping), and the final check is 0 errors and a stable 60 FPS.

What changed: I narrowed the scope from the full game to one slice, and every outside asset now gets credited in Credits.txt.

Needs attention: I still need to confirm how detailed "beautiful" needs to be (lighting and materials versus a clean whitebox) and which assets I'm allowed to use.

recap

What I understand: My goal is a beautiful, playable scene in Unity with a 3D player seen in third person. My plan is to build one small vertical slice of StimuLye, not the whole game. The slice is a dark opening, one looping ward corridor, and one Stimuli memory room that changes when the player picks up an item. I'll use Unity 6.6 with URP and the Third Person Starter Assets, so movement and the camera work from the start. The plan has five stages: Setup, NaN avatar, ward prefabs, the Stimuli mechanic, and a playtest. I test after every stage and keep a Project_LOG.txt, so I can see what changed if something breaks.

My job: I'm working solo, so I do every role. As technical lead I set up the project and Git. As the 3D artist I import and rig the NaN character from Mixamo. As the level designer I build the modular ward prefabs. As the programmer I write StimulusItem and RoomStabilizer. As QA I playtest for errors and frame rate. I also keep a credits file for every outside asset I use.

What I need to clarify: (1) Is the vertical slice small enough for me to finish and test alone? (2) Does "beautiful" mean polished lighting, fog and materials, or is a clean whitebox with good atmosphere enough? (3) Do I need a specific Unity version or template for this assignment, and will it still work if Unity updates packages mid-project? (4) Which free assets and audio am I allowed to use, and how should I credit them?

I'm ready to move on to the whiteboard.